

AXLERO SOLUTIONS

PROJECT DOCUMENT – ADVANCED DATA SCIENCE & ML ENGINEERING

Real-World Applications - Hands-on System Building

To continue pushing the boundaries of your Data Science and Machine Learning portfolio into 2026, this volume moves beyond standard predictive modeling and time-series forecasting. We focus on the bleeding edge of AI research: **Agentic Multi-Modal RAG**, **Privacy-Preserving Machine Learning (PPML)**, and **Multi-Agent Deep Reinforcement Learning**.

Here are three highly advanced, enterprise-grade ML projects for a senior-level portfolio.

Project 1 - "OmniBrain": Agentic Multi-Modal RAG Orchestrator

Domain: Large Language Models (LLMs) & Agentic AI

Problem Statement: Standard Retrieval-Augmented Generation (RAG) pipelines fail when documents contain complex multimodal data (like financial tables, embedded charts, and images) or when queries require multi-step reasoning across diverse data silos.

Use Case: A quantitative analyst feeds a 500-page corporate financial PDF into OmniBrain. Using a LangGraph supervisor agent, the system dynamically routes tasks. It parses tables using a Vision-Language Model (VLM), queries a vector database for semantic text, and uses a Text-to-SQL agent to query historical stock data [1.2.2]. The agents collaboratively synthesize the findings and output a highly accurate, cited investment memo, completely bypassing the hallucination risks of standard LLMs.

Key Modules:

- **Agentic Orchestrator (LangGraph):** Manages the state, memory, and routing of multiple specialized AI agents (Search Agent, SQL Agent, Vision Agent) [1.2.2].
- **Multi-Modal Retrieval (Qdrant / FAISS):** Stores both text chunks and image embeddings (using CLIP) for semantic similarity search.
- **Vision-Language Model (GPT-4o / LLaVA):** Extracts and reasons over visual charts and graphs embedded within the documents.
- **Evaluation & Guardrails (Langfuse & NeMo Guardrails):** Monitors the LLM outputs for toxicity, hallucinations, and ensures responses are strictly grounded in the retrieved context [1.2.2].

Week-wise Development Plan:

Week	AI Engineering (LangGraph, VLMs, Vector DBs)	Full-Stack Integration (FastAPI, Streamlit)
Week 1	Multi-Modal Ingestion: Build a pipeline to parse PDFs, chunk text, and extract images. Embed both modalities into a Qdrant vector database.	API Scaffolding: Build a FastAPI backend to handle asynchronous document uploads and querying.
Week 2	Agentic Architecture: Implement the LangGraph state machine. Define the "Supervisor" node that evaluates a user's query and routes it to the appropriate sub-agent.	Chat UI: Build a Streamlit interface capable of rendering the agent's "thought process" and displaying referenced images alongside text.
Mid-Project Review	Reasoning Audit: Prove the LangGraph supervisor can correctly decide between searching the vector database vs. executing a SQL query based on the prompt.	Vision Check: Ensure the VLM accurately extracts numerical data from a bar chart image retrieved from the database.
Week 3	Self-Correction Loop: Implement "Self-RAG." If an agent retrieves irrelevant data, it must recognize the failure, rewrite its own search query, and try again before responding to the user [1.2.2].	Guardrails: Integrate NeMo Guardrails to strictly block the AI from answering questions outside the scope of the provided documents.
Week 4	Evaluation Metrics: Integrate Langfuse to track token usage, latency, and LLM execution traces for observability [1.2.2].	Refine & Polish: Finalize the UI, adding citation links so users can click an AI claim and instantly view the exact PDF page and chart it referenced.
Final Review	A production-grade Agentic AI system capable of complex, autonomous reasoning over unstructured data.	A highly resilient, hallucination-resistant enterprise search tool.

Project 2 - "FedMed": Cross-Silo Federated Learning Engine

Domain: Privacy-Preserving Machine Learning (PPML) & Healthcare

Problem Statement: Training highly accurate ML models for rare diseases requires massive patient datasets. However, strict data privacy laws (HIPAA/GDPR) prevent hospitals from sharing raw patient data with centralized servers.

Use Case: Researchers at three different global hospitals collaborate to train a brain tumor segmentation model using MRI scans. Instead of pooling their private data, they deploy FedMed nodes. The central server sends the untrained PyTorch model to each hospital. The models train locally on private data, and only the *encrypted weight updates* are sent back and aggregated (using Secure Multi-Party Computation) to update the global model, preserving absolute patient privacy.

Key Modules:

- **Federated Learning Framework (Flower / PySyft):** Orchestrates the decentralized training loop, managing the communication between the central server and isolated client nodes.
- **Computer Vision Model (PyTorch / MONAI):** A 3D U-Net architecture designed specifically for segmenting medical imagery (e.g., MRI or CT scans).
- **Privacy & Encryption (TenSEAL):** Implements Homomorphic Encryption, allowing the central server to aggregate the model weights while they remain mathematically encrypted.
- **Training Dashboard (React / Recharts):** A monitoring UI showing the global model's convergence and accuracy metrics across different distributed epochs.

Week-wise Development Plan:

Week	PPML Engineering (PyTorch, Flower, TenSEAL)	Distributed Systems (gRPC, React)
Week 1	Centralized Baseline: Train a standard 3D U-Net model on a public MRI dataset (e.g., BraTS) to establish a baseline accuracy metric.	Node Scaffolding: Set up the Flower framework. Configure 3 distinct mock "Hospital Nodes" running on separate local ports.
Week 2	Federated Training Loop: Partition the dataset across the 3 nodes. Implement the logic for the central server to broadcast	Secure Communication: Implement gRPC with TLS certificates to ensure the network

Week	PPML Engineering (PyTorch, Flower, TenSEAL)	Distributed Systems (gRPC, React)
	weights, wait for local training, and aggregate the results (FedAvg).	traffic between the server and nodes is secure.
Mid-Project Review	Federated Audit: Prove the federated model converges and approaches the accuracy of the centralized baseline without ever exposing raw data to the central server.	Node Resilience: Ensure the training round survives if one of the 3 hospital nodes drops offline mid-epoch.
Week 3	Homomorphic Encryption: Integrate TenSEAL. Encrypt the PyTorch tensors on the client side before sending them to the server, requiring the server to perform mathematical aggregation on ciphertext.	Live Metrics: Stream the loss and accuracy metrics from the central aggregator to a WebSocket endpoint.
Week 4	Differential Privacy: Add controlled statistical noise to the weight updates before transmission to mathematically guarantee protection against model inversion attacks.	Refine & Polish: Build the React dashboard to visualize the training loss curve and the final MRI tumor segmentation masks.
Final Review	A masterclass in cryptography and decentralized deep learning.	A compliant, privacy-first healthcare AI architecture.

Project 3 - "SwarmRL": Multi-Agent Deep Reinforcement Learning Simulator

Domain: Autonomous Systems & Deep Reinforcement Learning

Problem Statement: Training a single agent to navigate an environment is well understood. However, coordinating a swarm of 50 autonomous agents to search a disaster zone dynamically without colliding—optimizing their coverage area collectively—is highly complex and fails with standard pathfinding algorithms.

Use Case: A disaster response coordinator uses SwarmRL to deploy simulated search drones. The backend uses Ray RLlib to run a Multi-Agent Proximal Policy Optimization (MAPPO) algorithm. The agents learn to collaborate in a decentralized manner, sharing spatial rewards to maximize the search area covered while heavily penalizing mid-air collisions. The React and WebGL dashboard renders the trained swarm executing a flawless, emergent search pattern in a 3D environment.

Key Modules:

- **Multi-Agent Environment (PettingZoo / OpenAI Gym):** A custom simulated continuous 3D space where multiple agents interact, observe their surroundings, and receive localized rewards.
- **Reinforcement Learning Engine (Ray RLLib):** A highly scalable distributed computing framework utilized to train the MAPPO algorithms across multiple CPU cores.
- **Deep Neural Networks (PyTorch):** The underlying actor-critic networks determining the continuous flight paths of the drones based on LiDAR-like state observations.
- **3D Simulation UI (React & Three.js):** A visualizer that ingests the live model inference coordinates and renders the swarm's flight paths and sensor coverage in 3D.

Week-wise Development Plan:

Week	Reinforcement Learning (Ray RLLib, PyTorch)	Simulation & Rendering (PettingZoo, Three.js)
Week 1	Environment Physics: Build the custom PettingZoo environment. Define the continuous action spaces (velocity, pitch, yaw) and observation spaces (distance to nearest neighbors).	3D Scaffolding: Set up the React and react-three-fiber canvas. Render a basic 3D topographical map with dummy spheres representing drones.
Week 2	Reward Engineering: Define the mathematical reward function. Reward agents for exploring unvisited coordinates (+1) and heavily penalize them for colliding with each other (-100).	State Broadcasting: Build a Python WebSocket server that pushes the exact X, Y, Z coordinates of all agents in the environment every step.
Mid-Project Review	Training Audit: Initiate training via Ray RLLib. Prove that after 1,000 episodes, the swarm learns to spread out rather than bunching together or flying out of bounds.	Rendering Check: Connect the WebSockets to Three.js and ensure the 3D visualizer smoothly animates the agents' movements.
Week 3	MAPPO Implementation: Upgrade from independent learning algorithms to Multi-Agent PPO, utilizing a centralized critic	Sensor Visuals: Enhance the Three.js rendering to show a semi-transparent "cone of vision"

Week	Reinforcement Learning (Ray RLlib, PyTorch)	Simulation & Rendering (PettingZoo, Three.js)
	during training and decentralized actors during inference.	extending from each drone, painting the ground green as it is searched.
Week 4	Curriculum Learning: Progressively increase the difficulty of the training environment by introducing dynamic obstacles and wind resistance.	Refine & Polish: Finalize the dashboard. Add analytics charts tracking the "Percentage of Map Explored" and "Total Collisions" over time.
Final Review	A highly complex integration of distributed Deep Learning and multi-agent systems.	A visually spectacular, interactive physics and AI simulation.

Common Features Across All Projects

- **Distributed Computing Architectures:** Moving beyond running models in a single Jupyter Notebook. These projects utilize distributed frameworks like Ray, Flower, and LangGraph to coordinate workloads across multiple agents, nodes, or CPU cores.
- **Advanced Neural Architectures:** Utilizing the absolute forefront of model architectures, including Vision-Language Models (VLMs), 3D U-Nets for medical imaging, and Multi-Agent Actor-Critic networks.
- **Complex Data Privacy & Guardrails:** Enterprise AI requires strict safety measures. These projects implement NeMo Guardrails for LLMs, Homomorphic Encryption, and Differential Privacy to ensure mathematical security.
- **Interactive Visualization:** Complex AI outputs are useless if they cannot be interpreted. These systems feature React and WebGL dashboards to render agent reasoning graphs, federated training metrics, and 3D drone simulations.